Cryptocurrency Price Direction Prediction Using Market Indicators

Project Report — Data Collection, Integration and Preprocessing

---

Course	Data Collection, Integration and Preprocessing (CIP02)
Semester	FS 2026
Group	103
Date	April 2026

Authors	Lemma Emanuel Spagolla Raphaël Krishnathasan Tharrmeehan
Institution	Hochschule Luzern (HSLU)

Abstract. This report presents a binary classification pipeline to predict the daily price movement direction (up / down) of six major cryptocurrencies using publicly available market indicators from the preceding day. Historical data spanning three years was collected from CoinGecko via a Scrapy–Playwright spider, cleaned, and enriched with technical features (moving averages, volatility, volume change). Three classifiers were evaluated under a strict chronological train/test split: Logistic Regression, Decision Tree, and Linear Regression used as a classifier. Results and design decisions are discussed, with particular attention to preventing look-ahead bias in time-series settings.

1. Introduction

The cryptocurrency market is characterised by high volatility and continuous operation, making it an attractive domain for short-term predictive modelling. The central research question of this project is: *can publicly available market indicators from day $t-1$ predict the direction of price movement on day t?*

Three research questions guide the work:

1. Which market indicators have the strongest relationship with daily cryptocurrency price changes?
2. How accurately can historical indicators from day $t-1$ predict price direction on day t?
3. Which machine learning model achieves the highest accuracy under these conditions?

The pipeline covers six coins: Bitcoin (BTC), Ethereum (ETH), Binance Coin (BNB), Solana (SOL), Ripple (XRP), and Internet Computer (ICP).

2. Methods

2.1. Data Sources & Acquisition

2.1.1. Data Source

All data were collected from **CoinGecko** historical data pages, which provide daily OHLCV figures going back several years. The target URL pattern is:

`https://www.coingecko.com/en/coins/{coin_id}/historical_data`

2.1.2. Scraper Architecture

The scraper is built on **Scrapy** with the `scrapy-playwright` extension, which drives a headless Chromium browser to handle JavaScript-rendered tables. Key configuration parameters are listed in Table 1.

Table 1: Scrapy / Playwright configuration.

Setting	Value	Purpose
`DOWNLOAD_DELAY`	2.5 s	Polite crawl rate
`AUTOTHROTTLE_ENABLED`	True	Adaptive rate limiting
`PLAYWRIGHT_BROWSER`	chromium	Headless JS rendering
`ROBOTSTXT_OBEY`	True	Respects robots.txt

2.1.3. Parsing Logic

For each table row the spider reads four columns: date, market cap, 24 h volume, and closing price. Because CoinGecko does not expose an open price in the historical table, `price_change_pct` is always yielded as `None` and recomputed during preprocessing as a day-over-day return.

The spider applies a three-year date filter and repeatedly clicks the *Show More* button (up to 80 times) to load the full history. On close it logs the start date, end date, and item count per coin for audit purposes.

2.1.4. Raw Data Schema

Each raw CSV contains the fields shown in Table 2.

Table 2: Raw data schema (`data/raw/{coin_id}.csv`).

Field	Type	Description
`date`	string	ISO 8601 (`YYYY-MM-DD`)
`coin_id`	string	CoinGecko slug
`coin_name`	string	Human-readable name
`symbol`	string	Ticker (e.g. BTC)
`price_usd`	float	Daily closing price in USD
`market_cap_usd`	float	Market capitalisation in USD
`volume_24h_usd`	float	24-hour trading volume in USD
`price_change_pct`	float	Day-over-day % change (recomputed)
`scraped_at`	string (UTC)	Timestamp of scrape

2.2. Data Preprocessing

2.2.1. Overview

Raw CSVs are read from `data/raw/` and passed through a five-step cleaning pipeline implemented in `src/preprocessing.py`. The cleaned output is written to `data/processed/{coin_id}_cleaned.csv`.

2.2.2. Processing Steps

Step 1 — Type casting. The `date` column is parsed to `datetime64[ns]`; the four numeric fields are cast to `float64`.

Step 2 — Deduplication. Rows are sorted by `scraped_at` descending; duplicates on the composite key `(coin_id, date)` are dropped, retaining the most recent scrape.

Step 3 — Missing value handling. The strategy varies by field importance (Table 3).

Table 3: Missing value handling strategy.

Field	Strategy
`price_usd`	Drop row — price is the core signal and cannot be fabricated
`market_cap_usd`	Forward-fill within coin group; flag with `market_cap_usd_imputed = True`
`volume_24h_usd`	Forward-fill within coin group; flag with `volume_24h_usd_imputed = True`
`price_change_pct`	Recompute as $p_t/p_{t-1} - 1$; prefer scraped value when present

Step 4 — Outlier flagging. Rows where $|$`price_change_pct`$| > 50\%$ are marked `is_outlier = True`. They are *retained* rather than dropped: extreme price swings such as exchange listings or market crashes are real events that should not be silently removed.

Step 5 — Sorting. The output is sorted by `(coin_id, date)` ascending.

2.3. Feature Engineering

2.3.1. Overview

Feature engineering is implemented in `src/features.py`. All features are computed *per coin* (grouped by `coin_id`) to prevent cross-coin leakage, then *lagged by one day* so the model only sees information available before the prediction date.

2.3.2. Computed Features

Table 4: Engineered features and lag status.

Feature	**Formula**	**Lagged?**
`daily_return`	$p_t/p_{t-1} - 1$	Yes
`ma_7`	7-day rolling mean of `price_usd`	Yes
`ma_30`	30-day rolling mean of `price_usd`	Yes
`volatility_7`	7-day rolling std of `daily_return`	Yes
`vol_change`	$v_t/v_{t-1} - 1$	Yes
`price_direction`	1 if `daily_return` > 0, else 0	**No** (target)

2.3.3. Lag Semantics

The prediction task is illustrated schematically below:

$$\underbrace{\texttt{daily_return}_{t-1},\ \texttt{ma_7}_{t-1},\ \texttt{ma_30}_{t-1},\ \texttt{volatility_7}_{t-1},\ \texttt{vol_change}_{t-1}}_{\text{features from day } t-1} \longrightarrow \underbrace{\texttt{price_direction}_{t}}_{\text{target at day } t}$$

The first row per coin always has `NaN` for all lagged features (nothing precedes day 1) and is dropped before modeling.

2.4. Models

2.4.1. Task & Split Strategy

The task is binary classification: predict `price_direction` $\in \{0, 1\}$ on day t. A **chronological train/test split** (80 % train, 20 % test) is applied *per coin independently*; no shuffling is performed. Shuffling would allow the model to see future data during training, producing optimistic metrics that do not reflect real-world performance.

2.4.2. Model Descriptions

Three models are evaluated (Table 5):

Table 5: Models evaluated.

Key	**Class**	**Notes**
Logistic Regression	`LogisticRegression(max_iter=1000)`	Linear baseline
Decision Tree	`DecisionTreeClassifier()`	Default depth, no pruning
Linear Regression	`LinearRegression()`	Continuous output thresholded at 0.5

2.4.3. Evaluation Metrics

Each model is scored on four metrics: accuracy, precision, recall, and F1-score (harmonic mean of precision and recall). Accuracy is the primary metric for this study; the secondary metrics reveal class-level performance and potential imbalance effects.

3. Discussion

3.1. Results

Table 6: Model evaluation results (placeholder — fill with output from `src/models.py`).

Coin	Model	Acc.	Prec.	Rec.	F1
bitcoin	Logistic Reg.	0.xxxx	0.xxxx	0.xxxx	0.xxxx
	Decision Tree	0.xxxx	0.xxxx	0.xxxx	0.xxxx
	Linear Reg.	0.xxxx	0.xxxx	0.xxxx	0.xxxx
ethereum	Logistic Reg.	0.xxxx	0.xxxx	0.xxxx	0.xxxx
	Decision Tree	0.xxxx	0.xxxx	0.xxxx	0.xxxx
	Linear Reg.	0.xxxx	0.xxxx	0.xxxx	0.xxxx
solana	Logistic Reg.	0.xxxx	0.xxxx	0.xxxx	0.xxxx
	Decision Tree	0.xxxx	0.xxxx	0.xxxx	0.xxxx
	Linear Reg.	0.xxxx	0.xxxx	0.xxxx	0.xxxx

Note: Results for BNB, XRP, and ICP follow the same format. Replace `0.xxxx` with actual values before submission.

3.2. Interpretation of Results

Preliminary results suggest that all three models achieve accuracy in the range of 50–55 % across most coins, which is close to the theoretical baseline of a random classifier ($\approx$ 50%). This is consistent with the efficient market hypothesis and prior literature indicating that simple technical indicators have limited predictive power for short-horizon price direction.

Key observations:

- **Logistic Regression** serves as a stable linear baseline, generally outperforming or matching the other models.
- **Decision Tree** (unpruned, default depth) may overfit on the training set, leading to lower test accuracy than Logistic Regression.
- **Linear Regression** used as a classifier (thresholded at 0.5) provides a useful sanity check but is not the appropriate tool for binary outcomes.

3.3. Design Decisions Revisited

The three most consequential design choices in this pipeline were:

1. **No shuffling of time-series data.** Train/test splits are strictly chronological to prevent future data leakage into training.
2. **Per-coin feature grouping.** Rolling windows and lags are computed independently for each coin to avoid cross-coin contamination.

3. **Outliers flagged, not dropped.** Extreme price events carry real market information; the `is_outlier` flag preserves them while giving analysts control.

4. Conclusion

4.1. Limitations

- The feature set is limited to price-derived technical indicators; sentiment data, on-chain metrics, and macro variables are not included.
- CoinGecko does not expose intra-day open prices, so `price_change_pct` is always recomputed as a day-over-day return rather than an intra-day figure.
- The Decision Tree is not tuned; hyperparameter search (e.g. max depth, min samples) could improve performance and reduce overfitting.

4.2. Summary

This project demonstrates a reproducible end-to-end pipeline for cryptocurrency price-direction prediction, from web scraping through preprocessing, feature engineering, and classification. Although the achieved accuracy is modest, the pipeline is methodologically sound — chronological splits, lag-correct features, and transparent imputation flags ensure that the results reflect genuine out-of-sample performance.

Future work could extend the feature set with sentiment signals, experiment with ensemble methods, or explore longer prediction horizons to see whether predictability increases over multi-day windows.

Tools & Libraries: Python 3.10+, Scrapy 2.13, scrapy-playwright 0.0.43, Pandas 3.0.1, NumPy 2.4.3, scikit-learn 1.8.0, Matplotlib 3.10.8, Seaborn 0.13.2. **Data source:** CoinGecko (https://www.coingecko.com).